

1.Abstract

Machine Learning (ML) has become an important technology for analyzing large amounts of data, identifying patterns, and making reliable predictions. The performance of a machine learning system depends significantly on the choice of an appropriate algorithm. This project focuses on the comparative analysis of different machine learning classification algorithms using the Diabetes Dataset. The Diabetes Dataset contains medical and demographic attributes of patients that can be used to predict whether a person is likely to have diabetes.

In this study, three supervised machine learning algorithms, namely Decision Tree, Gaussian Naive Bayes, and K-Nearest Neighbors (KNN), are applied to the Diabetes Dataset. The dataset is first examined and preprocessed to make it suitable for machine learning. The data is then divided into training and testing sets so that the models can be trained and evaluated effectively. Feature scaling is performed using StandardScaler to ensure that numerical features with different ranges do not negatively affect algorithms such as KNN.

2.Introduction

Machine Learning is a branch of Artificial Intelligence that enables computers to learn from data and make predictions or decisions without being explicitly programmed. It is widely used for classification, prediction, and data analysis in various fields, including healthcare.

In this project, the Diabetes Dataset is selected to study and compare different machine learning classification algorithms. The dataset contains several medical attributes such as glucose level, blood pressure, body mass index, age, insulin level, and other characteristics that can be used to predict diabetes. Three algorithms—Decision Tree, Naive Bayes, and K-Nearest Neighbors (KNN)—are applied to the dataset.

The performance of these algorithms is evaluated using accuracy, precision, recall, and F1-score. The results are compared using tables and graphs to determine which algorithm provides the best performance. This study helps in understanding the effectiveness of different machine learning algorithms for diabetes classification.

2.1 Overview of Diabetes Dataset

The Diabetes Dataset is a popular dataset used in machine learning for binary classification and healthcare-related data analysis. It contains medical diagnostic measurements of patients that can be used to predict whether a patient has diabetes.

The commonly used Pima Indians Diabetes Dataset contains 768 patient records and 8 input features, along with one target variable.

The important characteristics/features include:

- Pregnancies
- Glucose
- Blood Pressure
- Skin Thickness
- Insulin
- BMI
- Diabetes Pedigree Function
- Age

The target variable is:

- **0 – No Diabetes**
- **1 – Diabetes**

2.2 Role of Machine Learning in Diabetes Prediction

Machine Learning plays an important role in analyzing medical data and identifying patterns associated with diabetes. By using features such as glucose level, BMI, blood pressure, age, and insulin, machine learning algorithms can learn patterns from existing patient records.

In this project, Machine Learning is used to:

- Analyze patient medical characteristics.
- Identify patterns associated with diabetes.
- Classify patients into diabetic and non-diabetic categories.
- Compare the performance of different classification algorithms.
- Predict the diabetes outcome for new or unseen patient records.
- Evaluate model performance using accuracy, precision, recall, and F1-score.

3. System Requirements

The Diabetes Dataset Classification Using Machine Learning system requires basic hardware and software resources for data preprocessing, model training, execution, and performance evaluation. Since the Diabetes Dataset is relatively small, the project can be implemented efficiently on a standard laptop or desktop computer without requiring high-performance hardware.

3.1 Hardware Requirements

The hardware requirements for the proposed Diabetes Dataset Classification Using Machine Learning system are as follows:

- **Processor:** Intel Core i3 or higher.
- **RAM:** Minimum 4 GB RAM; 8 GB or more is recommended.
- **Hard Disk:** At least 10 GB of available storage space.
- **Operating System:** Windows 10/11, Linux, or macOS.
- **Keyboard and Mouse:** Standard keyboard and mouse.
- **Monitor:** Standard display with a resolution of 1366×768 or higher.
- **Internet Connection:** Required for accessing Google Colab, downloading datasets, and installing required libraries.

The Diabetes Dataset is relatively small and does not require a dedicated GPU or high-performance computing system. A standard laptop or desktop computer is sufficient for data preprocessing, model training, testing, visualization, and performance comparison.

3.2 Software Requirements

The software requirements for implementing the Diabetes Dataset Classification Using Machine Learning system are as follows:

- **Python 3.x:** Used as the primary programming language.
- **Google Colab:** Used as the development and execution environment.
- **Jupyter Notebook:** Can be used as an alternative environment.
- **Visual Studio Code:** Can also be used as an alternative development environment.

- **Pandas:** Used for loading, organizing, analyzing, and manipulating the Diabetes Dataset.
- **NumPy:** Used for numerical calculations and handling arrays.
- **Matplotlib:** Used for creating graphs and visualizing algorithm performance.
- **Seaborn:** Used for statistical visualization and confusion matrix heatmaps.
- **Scikit-learn:** Used for data preprocessing, machine learning algorithms, train-test splitting, and evaluation metrics.
- **CSV Dataset:** The Diabetes Dataset CSV file is used as the input dataset.

StandardScaler is used to standardize the numerical input features. This is particularly important for KNN because KNN uses distance calculations to classify observations.

4. Dataset Analysis and Preprocessing

The dataset and preprocessing stages are important parts of the Diabetes Dataset Classification system. Proper preprocessing ensures that the machine learning algorithms receive clean, organized, and suitable data for training and testing.

The commonly used Diabetes Dataset contains 768 patient records and 8 input features. The target variable contains two possible outcomes representing whether the patient has diabetes.

4.1 Dataset Overview

The dataset used in this project is the Diabetes Dataset, which is a structured dataset containing information about medical characteristics of patients. Each row represents a patient, while each column represents a specific medical attribute or the target outcome.

The dataset contains the following input features:

- Pregnancies
- Glucose
- Blood Pressure
- Skin Thickness
- Insulin

- BMI
- Diabetes Pedigree Function
- Age

The target variable is Outcome, which represents the diabetes classification.

4.2 Dataset Features

The Diabetes Dataset contains several numerical features that are used as input variables for predicting diabetes.

1. Pregnancies

Represents the number of times the patient has been pregnant.

2. Glucose

Represents the patient's plasma glucose concentration. This is an important feature for diabetes prediction.

3. Blood Pressure

Represents the patient's diastolic blood pressure measurement.

4. Skin Thickness

Represents the thickness of the skin fold measured at the triceps.

5. Insulin

Represents the patient's insulin level.

6. BMI

BMI stands for Body Mass Index and represents the relationship between a person's weight and height.

7. Diabetes Pedigree Function

This feature represents a function related to the patient's diabetes family history.

8. Age

Represents the age of the patient.

These features are provided as numerical values and are used by the machine learning algorithms to identify patterns associated with diabetes.

4.3 Target Variable

The target variable is **Outcome**. It represents whether the patient is classified as diabetic or non-diabetic.

The target values are:

- **0 – No Diabetes**
- **1 – Diabetes**

Since there are two possible target classes, this is a **binary classification problem**.

4.4 Dataset Shape and Structure

The commonly used Diabetes Dataset contains **768 rows and 9 columns**, including 8 input features and 1 target variable.

The dataset structure is examined using column names, data types, and statistical information. This helps in understanding the dataset and ensuring that the data is properly organized and suitable for machine learning.

4.5 Data Loading and Inspection

The Diabetes Dataset is loaded into the system using Pandas and stored in a DataFrame. The first few records are displayed using the head() function.

The column names, data types, dataset shape, and missing values are also examined. This inspection helps ensure that the dataset is properly prepared for the preprocessing and machine learning stages.

- **X = Input Features**
- **y = Target Variable**

The target variable is separated from the input features because it is the value that the machine learning models need to predict.

4.6 Train-Test Split

The Diabetes Dataset is divided into 80% training data and 20% testing data.

The training data is used to train the machine learning models, while the testing data is used to evaluate their performance on unseen patient records.

4.7 Feature Scaling

Feature scaling is performed to bring the numerical features of the Diabetes Dataset to a similar scale.

This is particularly important for K-Nearest Neighbors because KNN uses distance calculations. If one feature has a much larger numerical range than another, it can dominate the distance calculation.

5. Machine Learning Algorithms

Machine learning algorithms are the core components of the proposed Diabetes Classification system. They analyze patient medical characteristics and learn patterns that can be used to classify patients into diabetic and non-diabetic categories.

In this project, three supervised machine learning classification algorithms are used:

1. Decision Tree Classifier
2. Gaussian Naive Bayes
3. K-Nearest Neighbors (KNN)

The use of three different algorithms allows their performance to be compared using accuracy, precision, recall, F1-score, and confusion matrices.

5.1 Decision Tree Classifier

A Decision Tree Classifier is a supervised machine learning algorithm commonly used for classification problems. It works by dividing the dataset into smaller groups using a series of decision rules based on input features.

Working of Decision Tree

The Decision Tree starts with a root node containing the training data. It selects an appropriate feature and threshold to divide the data into different branches.

The splitting process continues until a suitable stopping condition is reached.

In this project, the Decision Tree uses patient characteristics such as glucose, BMI, age, blood pressure, and other features to learn patterns associated with diabetes.

5.2 Gaussian Naive Bayes

Gaussian Naive Bayes is a supervised classification algorithm based on Bayes' theorem. It assumes that the features are conditionally independent given the target class. It is called Gaussian Naive Bayes because it assumes that continuous numerical features follow a Gaussian or normal distribution within each class.

Working of Naive Bayes

When a new patient record is provided, the algorithm calculates the probability that the patient belongs to each possible class. The class with the highest probability is selected as the prediction.

In this project, Gaussian Naive Bayes uses patient features such as glucose, BMI, age, insulin, and blood pressure to estimate whether a patient is likely to belong to the diabetic or non-diabetic class.

5.3 K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm used for classification and regression problems. In classification, KNN predicts the class of a new data point based on the classes of its nearest neighboring data points. Unlike many other algorithms, KNN does not construct a complex model during training. Instead, it stores the training data and uses it when a new patient record needs to be classified.

Working of KNN

When a new patient record is provided, KNN calculates the distance between that record and the existing training records.

Importance of Feature Scaling in KNN

Feature scaling is especially important for KNN because it uses distance calculations.

For example, glucose values and diabetes pedigree values have different numerical ranges. Without scaling, features with larger numerical values may have a greater influence on distance calculations.

Therefore, StandardScaler is applied to the input features before training and testing.

5.4 Comparison of the Algorithms

The three algorithms used in this project use different approaches for classifying patients.

- **Naive Bayes** classifies patients using probability and Bayes' theorem.
- **KNN** classifies a new patient based on the classes of its nearest neighboring patient records.
- **Decision Tree** uses a sequence of feature-based decisions to classify patients.

Using these three algorithms provides a broader comparison of different machine learning approaches for Diabetes Dataset classification.

The performance of the models is compared using:

- Accuracy
- Precision
- Recall
- F1 Score
- Confusion Matrix
- Classification Report

The algorithm with the best overall performance based on the obtained evaluation results is considered the best-performing model for the Diabetes Dataset.

6. Performance Evaluation Metrics

Performance evaluation is an important stage in the Diabetes Dataset Classification system.

After training the Decision Tree, Naive Bayes, and KNN models, their performance is measured to determine how accurately they classify patients into diabetic and non-diabetic categories.

The main metrics used are:

- Accuracy
- Precision

- Recall
- F1 Score
- Confusion Matrix
- Classification Report

6.1 Accuracy

Accuracy measures the overall percentage of predictions that are correctly classified.

The formula is:

Accuracy = Number of Correct Predictions / Total Number of Predictions

For binary classification:

Accuracy = (TP + TN) / (TP + TN + FP + FN)

Where:

- **TP:** True Positive
- **TN:** True Negative
- **FP:** False Positive
- **FN:** False Negative

A higher accuracy indicates that the model correctly classifies a larger proportion of the test patient records.

6.2 Precision

Precision measures how accurately the model identifies samples predicted as belonging to a particular class.

The formula is:

Precision = TP / (TP + FP)

A high precision means that fewer patient records are incorrectly assigned to the predicted class.

6.3 Recall

Recall measures how effectively the model identifies the actual samples belonging to a particular class.

The formula is:

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

A high recall means that the model correctly identifies a larger proportion of actual cases belonging to the class.

In diabetes classification, recall can be particularly useful for understanding how effectively the model identifies patients belonging to the diabetes class.

6.4 F1 Score

The F1 Score combines precision and recall into a single performance measure.

The formula is:

$$\text{F1 Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

A higher F1 Score indicates a good balance between precision and recall.

In this project, the F1 Score is used along with accuracy, precision, and recall to provide a more complete evaluation of the three models.

6.5 Confusion Matrix

A confusion matrix is a table-like representation used to evaluate the predictions of a classification model.

Since the Diabetes Dataset contains two classes, the confusion matrix is a **2 × 2 matrix**.

The rows represent the actual classes, while the columns represent the predicted classes.

The confusion matrix helps identify:

1. Correctly classified non-diabetic patients.
2. Correctly classified diabetic patients.
3. Non-diabetic patients incorrectly classified as diabetic.
4. Diabetic patients incorrectly classified as non-diabetic.

The confusion matrix is displayed using a Seaborn heatmap.

6.6 Classification Report

The classification report provides a detailed summary of classification performance.

It generally includes:

- Precision
- Recall
- F1 Score
- Support

The report provides these values for:

- Class 0 – No Diabetes
- Class 1 – Diabetes

The support value represents the number of actual samples belonging to each class in the test dataset.

6.7 Importance of Evaluation Metrics in Diabetes Classification

Evaluation metrics are important in diabetes classification because accuracy alone may not provide a complete picture of model performance.

A model may achieve good overall accuracy while performing differently for diabetic and non-diabetic classes.

Therefore, precision, recall, F1 Score, and confusion matrix results are also considered.

Using multiple evaluation metrics helps identify the strengths and weaknesses of each model and provides a fair comparison between Decision Tree, Naive Bayes, and KNN.

6.8 Model Performance Comparison

The performance of Decision Tree, Naive Bayes, and KNN is compared using Accuracy, Precision, Recall, and F1 Score.

All three models are trained and tested using the same training and testing datasets to provide a fair comparison.

The importance of the metrics can be summarized as follows:

- **Accuracy:** Measures the overall percentage of correctly classified patient records.
- **Precision:** Measures how many samples predicted as a particular class actually belong to that class.
- **Recall:** Measures how effectively the model identifies actual samples belonging to each class.
- **F1 Score:** Provides a balance between precision and recall.

Therefore, all four metrics are considered when comparing the performance of the three machine learning algorithms.

The algorithm that achieves the best overall results is considered the best-performing model for Diabetes Dataset classification.

7.Implementation

The Diabetes Dataset Classification system is implemented using Python and the Scikit-learn library.

The implementation involves:

1. Loading the Diabetes Dataset.
2. Inspecting the dataset.
3. Separating features and target.
4. Splitting the data into training and testing sets.
5. Performing feature scaling.
6. Training three machine learning models.
7. Generating predictions.
8. Calculating evaluation metrics.
9. Creating confusion matrices.
10. Comparing model performance.

11. Identifying the best-performing algorithm.

7.1 Environment Setup

The system is implemented using Python and can be executed using Google Colab, Jupyter Notebook, or Visual Studio Code.

The major libraries used are:

- NumPy
- Pandas
- Matplotlib
- Seaborn
- Scikit-learn

Google Colab provides a convenient environment for executing the Python code without requiring extensive software installation.

The Diabetes Dataset is loaded into a Pandas DataFrame and prepared for model training and testing.

7.2 Model Implementation

Three machine learning classification algorithms are implemented:

- **Decision Tree Classifier**
- **Gaussian Naive Bayes**
- **K-Nearest Neighbors (KNN)**

7.3 Model Evaluation

The performance of the implemented models is evaluated using:

- Accuracy
- Precision
- Recall
- F1 Score

- Confusion Matrix
- Classification Report

The same training and testing datasets are used for all three algorithms to provide a fair comparison.

The model with the best overall performance across the evaluation metrics is considered the most suitable algorithm for the Diabetes Dataset.

7.4 Visualization

The confusion matrices of the three models are visualized using Seaborn heatmaps.

These visualizations help identify correctly and incorrectly classified patient records for each class.

A performance comparison bar graph is also created to compare:

- Accuracy
- Precision
- Recall
- F1 Score

of the three algorithms.

7.5 Implementation Code

The Python implementation used for data loading, preprocessing, model training, prediction, evaluation, and visualization is provided in the project.

The results obtained from the three machine learning algorithms are compared using suitable evaluation metrics. This comparison is used to identify the best-performing algorithm for the Diabetes Dataset.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classification_report)
```

```
file_path = ("/content/Diabetes_prediction.csv")
df = pd.read_csv(file_path)
```

```
print("\nFirst 5 rows of the dataset:")
display(df.head())
print("\nShape of the dataset:")
print(df.shape)
print("\nColumn names:")
print(df.columns.tolist())
```

```
print("\nDataset Information:")
df.info()
print("\nMissing Values:")
print(df.isnull().sum())
```

```
df.columns = df.columns.str.strip()
print("\nCleared Column Names:")
print(df.columns.tolist())
```

```
X = df.drop("Diagnosis", axis=1)
y = df["Diagnosis"]
print("\nFeatures:")
print(X.head())
print("\nTarget:")
print(y.head())
print("\nNumber of features:", X.shape[1])
print("Number of samples:", X.shape[0])
```

```
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.20,random_state=42,stratify=y)
print("\nTraining data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```

decision_tree = DecisionTreeClassifier(
    random_state=42
)
naive_bayes = GaussianNB()
knn = KNeighborsClassifier(
    n_neighbors=5
)

decision_tree.fit(
    X_train_scaled,
    y_train
)

naive_bayes.fit(
    X_train_scaled,
    y_train
)

knn.fit(
    X_train_scaled,
    y_train
)

y_pred_dt = decision_tree.predict(X_test_scaled)
y_pred_nb = naive_bayes.predict(X_test_scaled)
y_pred_knn = knn.predict(X_test_scaled)

```

```

dt_accuracy = accuracy_score(
    y_test,
    y_pred_dt
)

```

```

dt_precision = precision_score(
    y_test,
    y_pred_dt,
    average="weighted"
)

```

```

dt_recall = recall_score(
    y_test,
    y_pred_dt,
    average="weighted"
)

```

```

dt_f1 = f1_score(
    y_test,
    y_pred_dt,
    average="weighted"
)

```

```

nb_accuracy = accuracy_score(
    y_test,
    y_pred_nb
)

```

```

nb_precision = precision_score(
    y_test,
    y_pred_nb,
    average="weighted"
)

```

```

nb_recall = recall_score(
    y_test,
    y_pred_nb,
    average="weighted"
)

```

```

nb_f1 = f1_score(
    y_test,
    y_pred_nb,
    average="weighted"
)

```

```

knn_accuracy = accuracy_score(
    y_test,
    y_pred_knn
)

knn_precision = precision_score(
    y_test,
    y_pred_knn,
    average="weighted"
)

knn_recall = recall_score(
    y_test,
    y_pred_knn,
    average="weighted"
)

knn_f1 = f1_score(
    y_test,
    y_pred_knn,
    average="weighted"
)

results = pd.DataFrame({
    "Algorithm": [
        "Decision Tree",
        "Naive Bayes",
        "KNN"
    ],
    "Accuracy": [
        dt_accuracy,
        nb_accuracy,
        knn_accuracy
    ],
    "Precision": [
        dt_precision,
        nb_precision,
        knn_precision
    ],
    "Recall": [
        dt_recall,
        nb_recall,
        knn_recall
    ],
    "F1 Score": [
        dt_f1,
        nb_f1,
        knn_f1
    ]
})
print("\n-----")
print("PERFORMANCE COMPARISON")
print("-----")
display(results)

```

```

results_percentage = results.copy()
results_percentage[
    ["Accuracy", "Precision", "Recall", "F1 Score"]
] = (
    results_percentage[
        ["Accuracy", "Precision", "Recall", "F1 Score"]
    ] * 100
).round(2)
print("\n=====")
print("PERFORMANCE COMPARISON IN PERCENTAGE")
print("=====")

display(results_percentage)

print("\n=====")
print("CLASSIFICATION REPORT - DECISION TREE")
print("=====")
print(
    classification_report(
        y_test,
        y_pred_dt
    )
)
print("\n=====")
print("CLASSIFICATION REPORT - NAIVE BAYES")
print("=====")

print(
    classification_report(
        y_test,
        y_pred_nb
    )
)
print("\n=====")
print("CLASSIFICATION REPORT - KNN")
print("=====")

print(
    classification_report(
        y_test,
        y_pred_knn
    )
)

cm_dt = confusion_matrix(
    y_test,
    y_pred_dt
)
plt.figure(figsize=(4, 4))
sns.heatmap(
    cm_dt,
    annot=True,
    fmt="d",
    cmap="Blues"
)
plt.title("Confusion Matrix - Decision Tree")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.show()

cm_nb = confusion_matrix(
    y_test,
    y_pred_nb
)
plt.figure(figsize=(4, 4))
sns.heatmap(
    cm_nb,
    annot=True,
    fmt="d",
    cmap="Blues"
)
plt.title("Confusion Matrix - Naive Bayes")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.show()

```

```

cm_knn = confusion_matrix(
    y_test,
    y_pred_knn
)
plt.figure(figsize=(4, 4))
sns.heatmap(
    cm_knn,
    annot=True,
    fmt="d",
    cmap="Blues"
)
plt.title("Confusion Matrix - KNN")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.show()

results_percentage.set_index(
    "Algorithm"
)[
    ["Accuracy", "Precision", "Recall", "F1 Score"]
].plot(
    kind="bar",
    figsize=(6, 6)
)
plt.title(
    "Performance Comparison of ML Algorithms"
)
plt.xlabel("Algorithms")
plt.ylabel("Performance (%)")
plt.xticks(rotation=0)
plt.ylim(0, 100)
plt.legend(
    title="Evaluation Metrics"
)
plt.grid(
    axis="y"
)
plt.show()

best_index = results["F1 Score"].idxmax()
best_algorithm = results.loc[
    best_index,
    "Algorithm"
]
best_accuracy = results.loc[
    best_index,
    "Accuracy"
]
best_precision = results.loc[
    best_index,
    "Precision"
]
best_recall = results.loc[
    best_index,
    "Recall"
]
best_f1 = results.loc[
    best_index,
    "F1 Score"
]

```

```

print("\n")
print("=====")
print("              FINAL RESULT")
print("=====")
print(
    "\nBest Performing Algorithm:",
    best_algorithm
)
print(
    "Accuracy:",
    round(best_accuracy * 100, 2),
    "%"
)
print(
    "Precision:",
    round(best_precision * 100, 2),
    "%"
)
print(
    "Recall:",
    round(best_recall * 100, 2),
    "%"
)
print(
    "F1 Score:",
    round(best_f1 * 100, 2),
    "%"
)

print("\n=====")
print("CONCLUSION")
print("=====")

print(
    f"\nAmong Decision Tree, Naive Bayes and KNN, "
    f"{best_algorithm} performed the best based on "
    f"the highest F1 Score."
)

```

8. Results and Discussion

The Results and Discussion chapter presents the performance of the machine learning models used for Diabetes Dataset classification.

Three classification algorithms were used in this project:

1. Gaussian Naive Bayes
2. K-Nearest Neighbors (KNN)
3. Decision Tree Classifier

8.1 Dataset Preview

The first five records of the Diabetes Dataset were displayed using the `df.head()` function. This provides an initial view of the dataset and helps verify that the diabetes data has been loaded correctly.

First 5 rows of the dataset:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Diagnosis
0	2	115.863387	56.410731	24.336736	94.385783	26.455940	0.272082	20.100494	0
1	2	92.490122	70.615520	23.443591	138.652426	23.910167	0.665160	44.912281	0
2	1	88.141469	63.282618	23.404364	149.358082	21.948250	0.676022	48.247873	1
3	2	108.453101	67.793632	20.751580	108.751638	24.209304	0.289636	42.749868	0
4	1	127.849443	94.725685	22.603078	25.269987	32.997477	0.601315	32.797789	0

Figure 8.1: Dataset Preview – First Five Rows

The displayed output shows the first five patient records along with their medical features used for diabetes classification.

8.2 Dataset Shape and Testing Data

The shape of the Diabetes Dataset was checked using:

The dataset contains the patient records and corresponding attributes.

The data is divided into:

- **80% Training Data**
- **20% Testing Data**

Shape of the dataset:
(1000, 9)

Figure 8.2: Dataset Shape

8.3 Model Performance Results

The three machine learning algorithms—Gaussian Naive Bayes, K-Nearest Neighbors, and Decision Tree—were trained using the Diabetes Dataset and tested using the same testing data.

The performance of all three models was evaluated using:

- Accuracy
- Precision
- Recall
- F1 Score

The calculated results were combined into a single performance comparison table.

```
=====
PERFORMANCE COMPARISON
=====
```

	Algorithm	Accuracy	Precision	Recall	F1 Score
0	Decision Tree	0.530	0.542463	0.530	0.535832
1	Naive Bayes	0.695	0.636894	0.695	0.578881
2	KNN	0.635	0.573912	0.635	0.591139

```
=====
PERFORMANCE COMPARISON IN PERCENTAGE
=====
```

	Algorithm	Accuracy	Precision	Recall	F1 Score
0	Decision Tree	53.0	54.25	53.0	53.58
1	Naive Bayes	69.5	63.69	69.5	57.89
2	KNN	63.5	57.39	63.5	59.11

Figure 8.3: Performance Comparison Table

8.4 Classification Report

Decision Tree

```
=====
CLASSIFICATION REPORT - DECISION TREE
=====
```

	precision	recall	f1-score	support
0	0.67	0.64	0.65	139
1	0.25	0.28	0.27	61
accuracy			0.53	200
macro avg	0.46	0.46	0.46	200
weighted avg	0.54	0.53	0.54	200

The classification report for the Decision Tree displays the precision, recall, F1 Score, and support for Class 0 and Class 1.

Naive Bayes

```

=====
CLASSIFICATION REPORT - NAIVE BAYES
=====
              precision    recall  f1-score   support

     0       0.70        0.99        0.82        139
     1       0.50        0.02        0.03         61

 accuracy          0.69          200
 macro avg         0.60          200
 weighted avg      0.64          200

```

The classification report for Gaussian Naive Bayes displays the precision, recall, F1 Score, and support for the two diabetes classes.

KNN

```

=====
CLASSIFICATION REPORT - KNN
=====
              precision    recall  f1-score   support

     0       0.69        0.85        0.76        139
     1       0.30        0.15        0.20         61

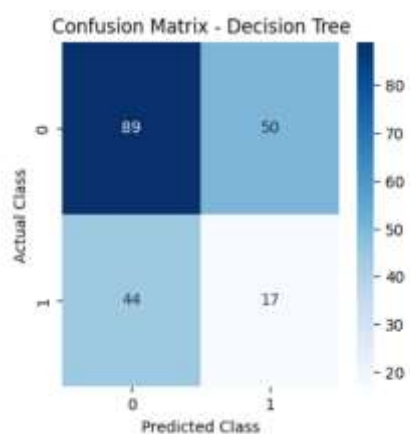
 accuracy          0.64          200
 macro avg         0.50          200
 weighted avg      0.57          200

```

The classification report for KNN displays the precision, recall, F1 Score, and support for Class 0 and Class 1.

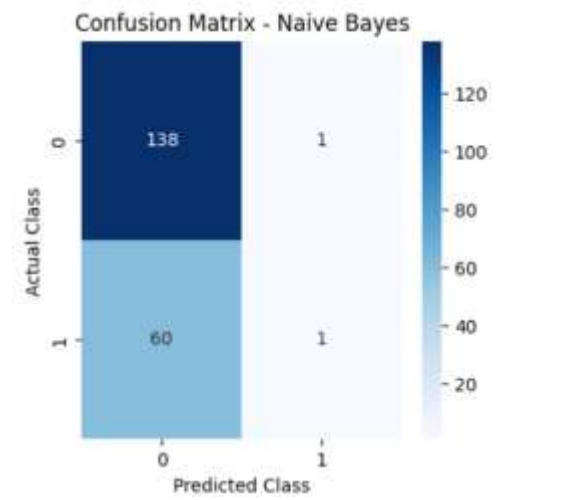
8.5 Confusion Matrix

Decision Tree



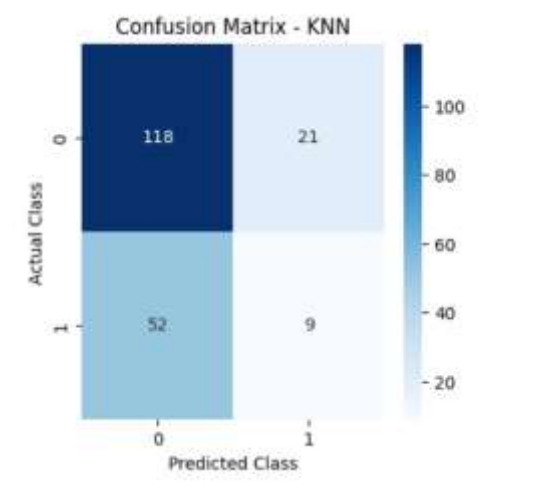
The Decision Tree confusion matrix represents the correctly and incorrectly classified patient records.

Naive Bayes



The Naive Bayes confusion matrix represents the classification results for diabetic and non-diabetic patients.

KNN



The KNN confusion matrix represents the classification results obtained from the five-nearest-neighbor approach.

8.6 Discussion of Results

The performance of the Gaussian Naive Bayes, K-Nearest Neighbors, and Decision Tree models was evaluated using Accuracy, Precision, Recall, and F1 Score. All three models were trained and tested using the same Diabetes Dataset, providing a fair basis for comparison.

The **Accuracy** indicates the overall percentage of patient records correctly classified by each model. **Precision** shows how accurately the models classify records into the predicted diabetes classes. **Recall** measures how effectively the models identify the actual samples belonging to each class. The **F1 Score** provides a balance between precision and recall.

The confusion matrices and classification reports provide additional information about correctly and incorrectly classified patient records. Based on the comparison of the evaluation metrics, the algorithm that achieves the best overall performance is selected as the best-performing model for the Diabetes Dataset classification task.

The final model selection is based on the actual results obtained from the implementation rather than considering only one evaluation metric.

9.Conclusion

The Diabetes Dataset Classification Using Machine Learning project successfully demonstrated the application of machine learning classification algorithms to predict diabetes based on patient medical characteristics. Three algorithms—Gaussian Naive Bayes, K-Nearest Neighbors (KNN), and Decision Tree Classifier—were implemented and evaluated using the same training and testing data.

The performance of the models was compared using Accuracy, Precision, Recall, F1 Score, Confusion Matrix, and Classification Report. The results provided a detailed understanding of how each algorithm performs in classifying patients into diabetic and non-diabetic categories.

The project also demonstrated the importance of data preprocessing, feature scaling, model training, and performance evaluation in developing a reliable machine learning classification system. The comparison of the algorithms helps identify the model that provides the best overall performance for the given Diabetes Dataset.

Based on the comparison of the obtained results, the algorithm with the best overall performance can be selected as the most suitable model for the Diabetes Dataset. Overall, the project demonstrates that machine learning algorithms can be effectively used for diabetes classification based on patient medical characteristics. The comparison of Decision Tree, Naive Bayes, and KNN provides a useful understanding of their performance and helps in selecting an appropriate classification algorithm.

In this study, three supervised machine learning algorithms are implemented and compared: Decision Tree Classifier, Gaussian Naive Bayes, and K-Nearest Neighbors (KNN). These algorithms were selected because they use different approaches to classification and provide a useful basis for comparing machine learning techniques. The Decision Tree algorithm uses a tree-based decision-making structure to classify patient records. Gaussian Naive Bayes uses probability and Bayes' theorem to determine the most likely class. K-Nearest Neighbors classifies a patient based on the classes of the nearest training samples.